

POCKETSMART AI

Your Smart Budget & Recommendation and Assistant

PROJECT DOCUMENTATION

A Smart Budget Management and Recommendation System

Technology: Python and HTML

✓ **Project Report**

CONTENTS

1. 1. Project Description
2. 2. Problem Statement
3. 3. Objectives
4. 4. Scope of the Project
5. 5. Project Scenarios
6. 6. System Requirements
7. 7. Technical Architecture
8. 8. Project Workflow
9. 9. System Flowchart
10. 10. Development Milestones
11. 11. Core Python Functionality
12. 12. Frontend Development
13. 13. Data Flow Diagram
14. 14. Use Case Diagram
15. 15. Module Description
16. 16. Input and Output Design
17. 17. Sample Working
18. 18. Graphical Representation
19. 19. Testing and Test Cases
20. 20. Advantages
21. 21. Limitations
22. 22. Future Enhancements
23. 23. Conclusion

1. Project Description

PocketSmart AI: Your Smart Budget & Recommendation and Assistant is a budget-oriented application designed to help a user understand how much money is available within a monthly budget after expenses. The application focuses on a simple and clear process: the user provides a name, a monthly budget and the total amount spent, after which the program calculates the remaining balance and produces a recommendation.

The current implementation supplied for the project uses Python for the main calculation and decision logic. The Python program accepts the user's values through input statements, performs the balance calculation, displays a budget summary and checks the result using conditional statements. A separate HTML file provides the basic PocketSmart AI interface and project introduction.

The project is intentionally based on the programs supplied for PocketSmart AI. Therefore, features such as database storage, live bank connectivity and external generative-AI services are not described as currently implemented. They are included later as possible enhancements.

Key Characteristics

- Simple user input and easy-to-understand processing.
- Automatic calculation of remaining balance.
- Condition-based budget recommendation.
- Clear summary of name, budget, expenses and balance.
- HTML interface for presenting the project identity.
- Suitable foundation for further web-based budget features.

2. Problem Statement

Managing a monthly budget manually can make it difficult for a user to quickly understand how much money remains after expenses. Repeated calculations may also lead to mistakes, especially when expenses change during the month.

PocketSmart AI addresses this basic problem by accepting the monthly budget and total expenses and performing the subtraction automatically. The result is converted into an easy-to-understand recommendation. This gives the user a quick view of whether the available budget remains positive, has been fully used, or has been exceeded.

Problem Identification

- Manual calculation of remaining budget.
- Difficulty in quickly interpreting spending status.
- Lack of a simple summary after entering expenses.
- Need for a basic recommendation based on the calculated balance.

3. Objectives

- 24. To design a simple budget calculation application.
- 25. To accept monthly budget and expense values from the user.
- 26. To calculate the remaining balance automatically.
- 27. To apply conditional logic for budget recommendations.
- 28. To display the user's budget information in a structured summary.
- 29. To provide a saving suggestion when the remaining balance is positive.
- 30. To create a basic HTML interface for the PocketSmart AI project.
- 31. To provide a foundation that can later be expanded into a complete budget assistant.

Expected Outcome

After entering valid values, the system should calculate the remaining balance correctly and display a recommendation corresponding to the balance condition.

4. Scope of the Project

The current scope covers basic monthly budget calculation and recommendation. It is designed around three main inputs: user name, monthly budget and total expenses.

Included in Current Scope

- Name input.
- Monthly budget input.
- Total expense input.
- Balance calculation.
- Positive, zero and negative balance conditions.
- Budget summary output.
- Basic saving/recommendation message.
- Basic HTML project interface.

Outside Current Scope

- Automatic bank-account access.
- Real-time transaction import.
- Database-based expense history.
- Authentication and user accounts.
- Actual external AI model/API integration.
- Automatic financial transactions.

5. Project Scenarios

Scenario 1 – Entering Budget Information

The user starts PocketSmart AI and enters their name, monthly budget and total expenses. The values become the input for the budget calculation.

Scenario 2 – Remaining Balance is Positive

If the monthly budget is greater than total expenses, the calculated balance is positive. The program informs the user that they are within their budget and shows the amount that can be saved.

Scenario 3 – Remaining Balance is Zero

If expenses are exactly equal to the monthly budget, the balance becomes zero. The program indicates that the full budget has been used.

Scenario 4 – Expenses Exceed Budget

If total expenses are greater than the monthly budget, the balance becomes negative. The program identifies that the budget has been exceeded.

Scenario 5 – Reviewing the Budget Summary

After calculation, the user can see the name, monthly budget, total expenses, remaining balance and recommendation together as a single summary.

6. System Requirements

6.1 Software Requirements

Requirement	Technology	Purpose
Programming Language	Python	Core calculation and decision logic
Frontend	HTML	Basic user interface
Editor	VS Code / IDLE / similar	Code development
Browser	Any modern browser	Viewing the HTML interface
Operating System	Windows / Linux / macOS	Execution environment

6.2 Hardware Requirements

- Basic computer or laptop.
- Keyboard for entering values.
- Display for viewing the result.
- Internet connection is not required for the current local Python calculation.

7. Technical Architecture

The architecture separates the project into user interaction, frontend presentation, Python processing, calculation, recommendation and output stages. The supplied HTML file represents the frontend identity, while the supplied Python program performs the current processing.

PocketSmart AI – System Architecture

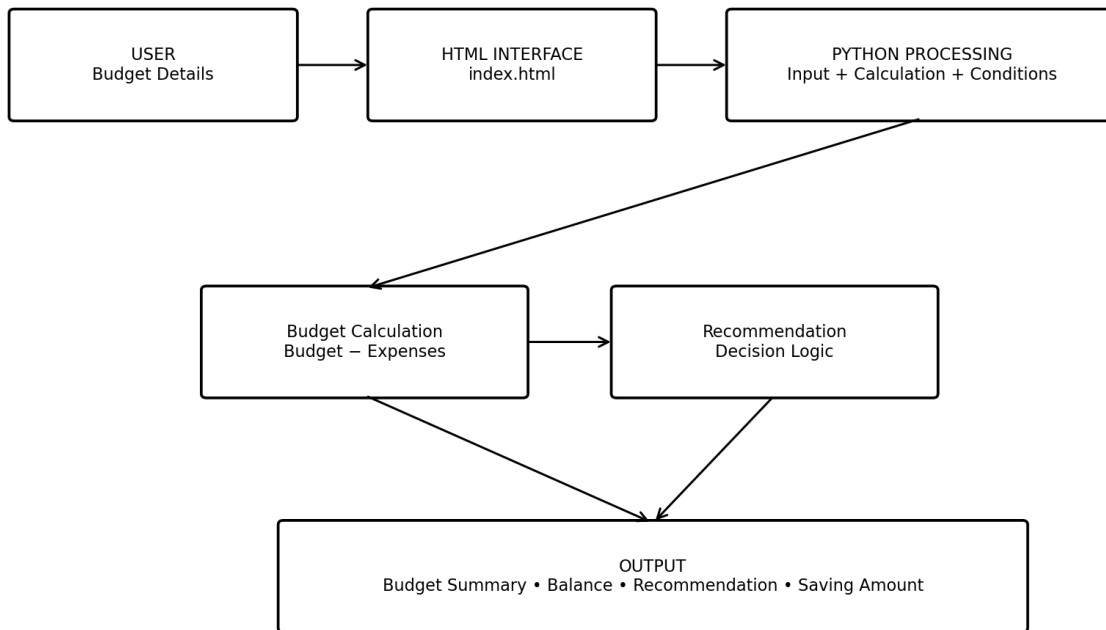


Figure 1. System Architecture

7.1 Component Responsibilities

- User Layer: supplies name, monthly budget and expenses.
- HTML Layer: presents the PocketSmart AI project interface.
- Python Processing Layer: reads and processes values.
- Calculation Layer: calculates Budget – Expenses.

Recommendation Layer: evaluates the balance using conditional statements.

Output Layer: displays the final budget summary.

8. Project Workflow

The complete logical workflow of the current implementation is described below.

32. Application starts.
33. User enters their name.
34. User enters the monthly budget.
35. User enters total expenses.
36. Python converts budget and expense values into numeric values.
37. Remaining balance is calculated using $\text{Budget} - \text{Expenses}$.
38. The program checks whether balance is greater than, equal to, or less than zero.
39. A recommendation is selected.
40. The budget summary is displayed.
41. The process ends.

9. System Flowchart

The flowchart represents the detailed decision process used by PocketSmart AI. The positive, zero and negative balance conditions are separated so that the user receives a meaningful status message.

PocketSmart AI – Detailed System Flowchart

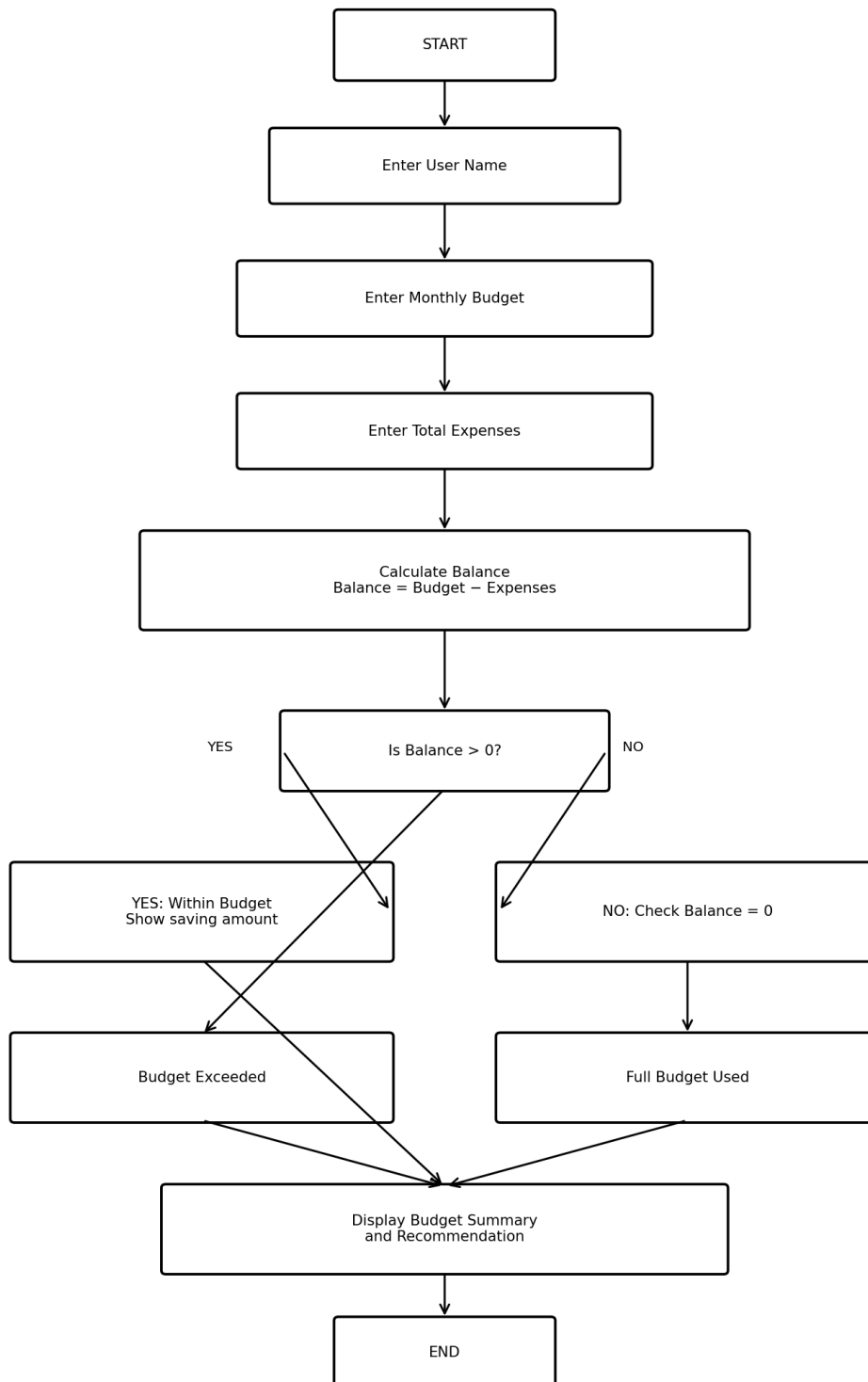


Figure 2. Detailed System Flowchart

10. Development Milestones

Milestone 1 – Requirement Analysis

- Identify the need for a simple budget calculator.
- Identify required inputs and outputs.
- Define the balance formula and recommendation conditions.

Milestone 2 – Python Program Development

- Create input statements.
- Convert numeric inputs using float().
- Implement the balance formula.
- Create the budget summary output.

Milestone 3 – Recommendation Logic

- Implement if, elif and else conditions.
- Handle positive balance.
- Handle zero balance.
- Handle negative balance.

Milestone 4 – HTML Interface

- Create index.html.
- Add PocketSmart AI title and description.
- Prepare a base page for future web integration.

Milestone 5 – Testing

- Test normal positive-balance values.
- Test exact budget usage.
- Test budget-exceeded values.
- Verify displayed values and recommendation.

11. Core Functionality – Python Program

The Python program is the main computational component. It uses `input()` for user interaction and `float()` to support numeric budget and expense values. The balance is calculated and the result is evaluated with conditional statements.

11.1 Program Logic

```
name = input("Enter your name: ")
    budget = float(input("Enter your monthly budget: "))
    expense = float(input("Enter your total expenses: "))

    balance = budget - expense

    print("----- Budget Summary -----")
    print("Name:", name)
    print("Monthly Budget: ₹", budget)
    print("Total Expenses: ₹", expense)
    print("Remaining Balance: ₹", balance)

    if balance > 0:
        print("Recommendation: You are within your budget.")
        print("You can save ₹", balance)
    elif balance == 0:
        print("Recommendation: You have used your full budget.")
    else:
        print("Recommendation: You have exceeded your budget.")
```

11.2 Explanation of Important Statements

`input()`

Reads the user's name, monthly budget and total expenses.

`float()`

Converts budget and expense text into numbers so arithmetic can be performed.

balance = budget - expense

Calculates the amount remaining from the monthly budget.

if balance > 0

Checks whether some budget remains.

elif balance == 0

Checks whether the full budget has been used.

else

Handles the case where expenses exceed the budget.

12. Frontend Development – HTML Program

The supplied HTML program provides the initial presentation layer for PocketSmart AI. It contains the HTML document structure, page title, project heading, subtitle and introductory text.

```
<!DOCTYPE html>
<html>
<head>
  <title>PocketSmart AI</title>
</head>
<body>
  <h1>PocketSmart AI</h1>
  <h2>Skill Wallet</h2>
  <p>Manage your skills easily.</p>
</body>
</html>
```

HTML Elements Used 12.1

<html> defines the HTML document.

<head> contains document information.

<title> sets the browser tab title.

<body> contains visible page content.

<h1> displays the main project heading.

<h2> displays the secondary heading.

<p> displays descriptive text.

12.2 Current Frontend Status

The current HTML page is a basic interface and is not yet connected to the Python input and calculation logic. Connecting the two is a future development step.

13. Data Flow Diagram

The Level 0 DFD represents the flow of user data into the PocketSmart AI processing system and the return of calculated information and recommendations.

PocketSmart AI – Data Flow Diagram (Level 0)

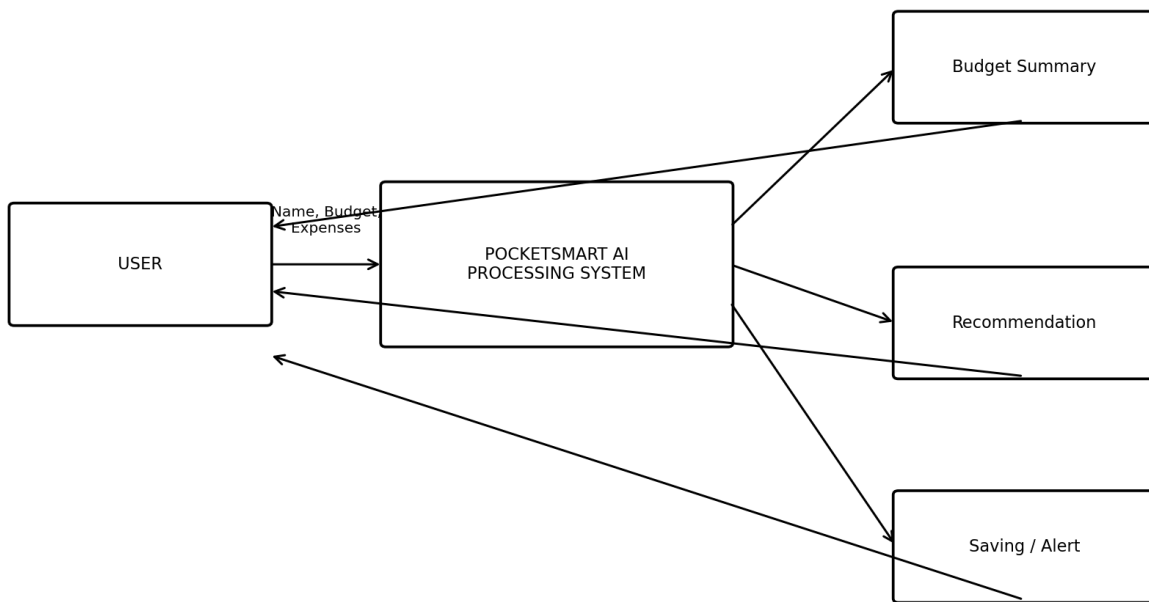


Figure 3. Data Flow Diagram – Level 0

13.1 Data Items

- Name – identifies the user.
- Monthly Budget – amount available for the month.
- Total Expenses – amount already spent.
- Remaining Balance – calculated result.
- Recommendation – message based on the balance condition.

14. Use Case Diagram

The use case diagram shows the main interactions between the user and PocketSmart AI. The user provides the required budget information and receives the processed result.

PocketSmart AI – Use Case Diagram

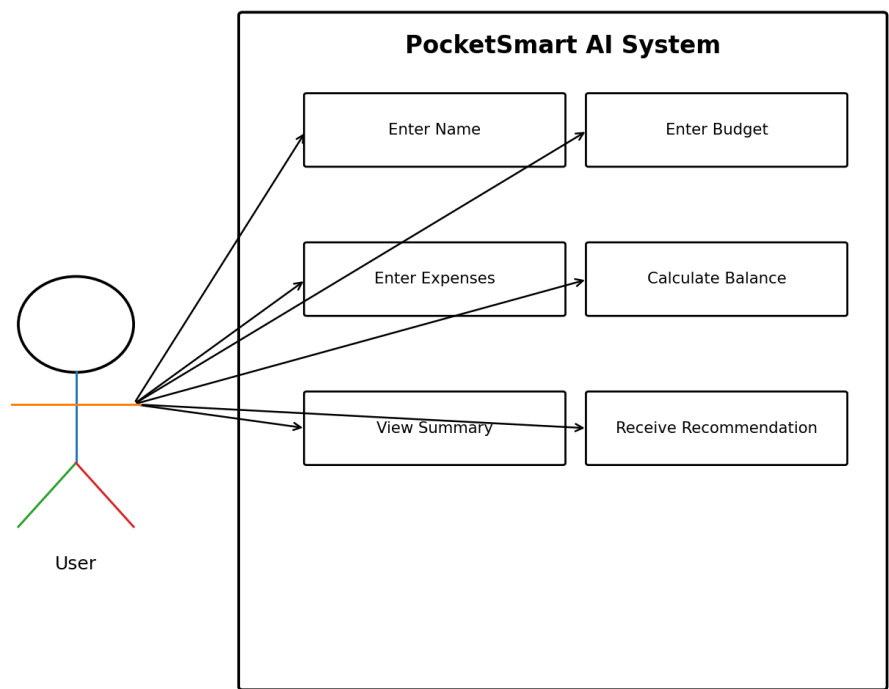


Figure 4. Use Case Diagram

15. Module Description

15.1 User Input Module

Collects the user's name, monthly budget and total expenses. It forms the starting point of the processing sequence.

15.2 Data Conversion Module

Converts budget and expense input into floating-point values so mathematical calculations can be performed.

15.3 Budget Calculation Module

Calculates remaining balance using the formula: $\text{Remaining Balance} = \text{Monthly Budget} - \text{Total Expenses}$.

15.4 Recommendation Module

Uses conditional statements to determine whether the user is within the budget, has used the full budget or has exceeded the budget.

15.5 Summary Output Module

Displays the user's name, monthly budget, total expenses, remaining balance and recommendation.

15.6 HTML Presentation Module

Provides the basic PocketSmart AI project page and can act as the starting point for a browser-based interface.

Module Sequence

**User Input → Data Conversion → Calculation → Decision → Recommendation → Summary
Output**

16. Input and Output Design

16.1 Input Design

Input	Type	Purpose
Name	Text	Identify the user
Monthly Budget	Float	Specify available monthly budget
Total Expenses	Float	Specify total spending

16.2 Output Design

Output	Description
Name	Displays entered user name
Monthly Budget	Displays entered budget
Total Expenses	Displays entered expenses
Remaining Balance	Displays budget minus expenses
Recommendation	Displays the condition-based message
Saving Amount	Displays positive remaining balance when applicable

17. Sample Working

The following example is based on the values visible in the supplied PocketSmart AI demonstration.

Field	Sample Value
Name	An Naabikha
Monthly Budget	₹20,000
Total Expenses	₹3,500
Balance	₹16,500

Calculation

$$\text{Remaining Balance} = \text{Monthly Budget} - \text{Total Expenses}$$

$$\text{₹20,000} - \text{₹3,500} = \text{₹16,500}$$

Because the balance is positive, the current program displays a within-budget recommendation and indicates that the remaining amount can be saved.

18. Graphical Representation

18.1 Budget Comparison Graph

This graph visualizes the sample budget, total expenses and remaining balance. It is based on the example values shown in the project demonstration.

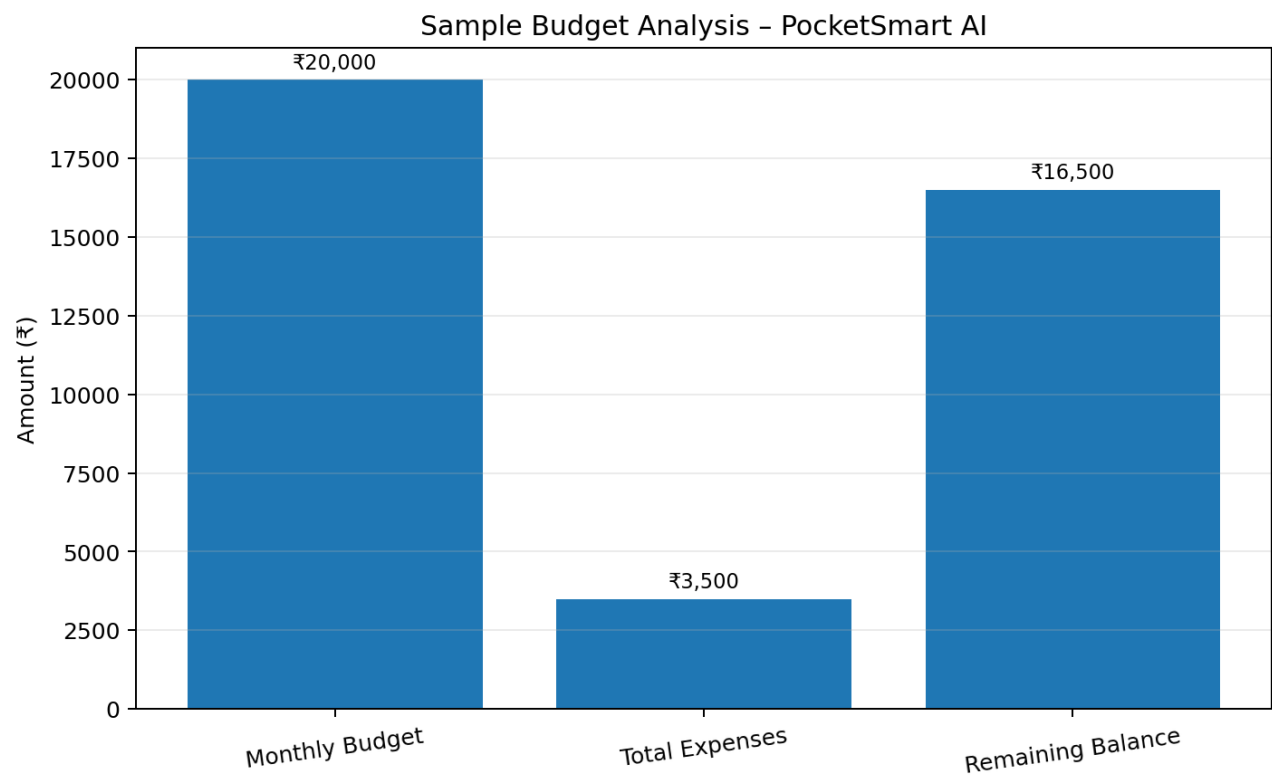


Figure 5. Sample Budget Comparison

18.2 Illustrative Expense Distribution

The following chart is an illustrative example showing how total expenses could be divided into categories in a future version. These category values are not produced by the current Python program.

Illustrative Expense Category Distribution

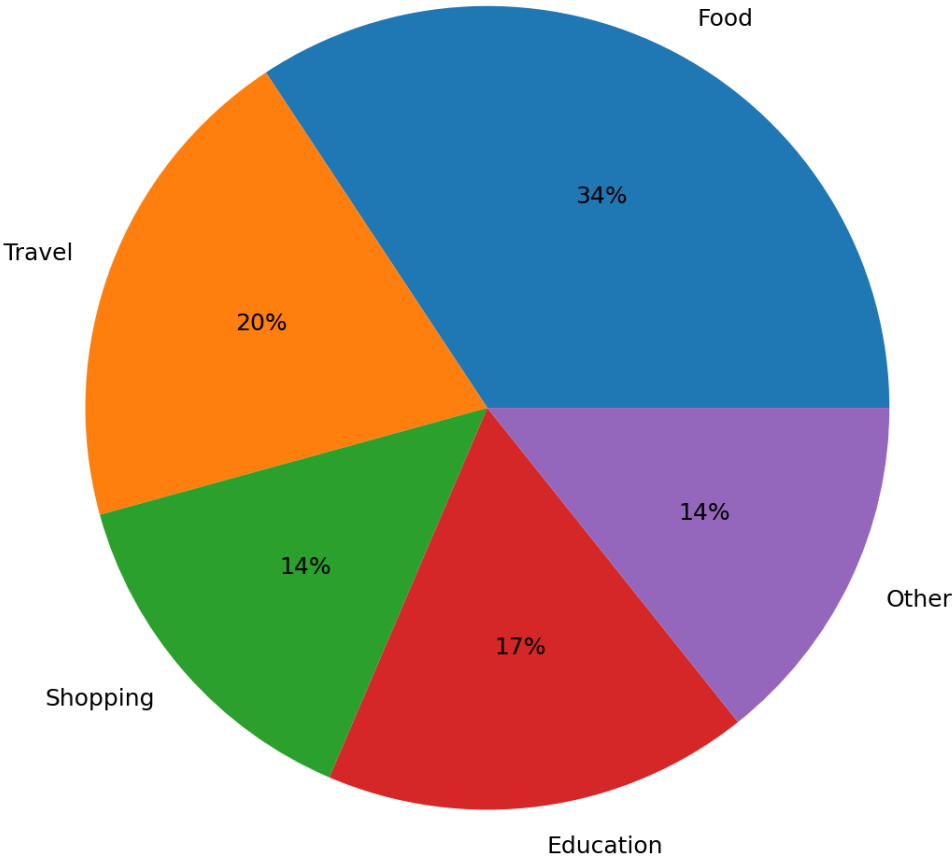


Figure 6. Illustrative Expense Category Distribution

19. Testing and Test Cases

Testing verifies that the calculation and conditional recommendation work correctly for different budget situations.

Test ID	Condition	Budget	Expenses	Expected Result
TC01	Positive balance	₹20,000	₹3,500	₹16,500; within-budget message
TC02	Zero balance	₹10,000	₹10,000	₹0; full-budget message
TC03	Negative balance	₹10,000	₹12,000	-₹2,000; exceeded-budget message
TC04	Decimal input	₹15,500.50	₹4,250.25	₹11,250.25 remaining
TC05	Small budget	₹1,000	₹250	₹750 remaining

Validation Points

- Check that budget and expense values are numeric.
- Check the subtraction result.
- Check positive/zero/negative branches.
- Check the displayed recommendation.
- Check that the HTML page loads without syntax errors.

20. Advantages

- Easy to understand and operate.
- Fast calculation of remaining balance.
- Reduces simple manual calculation errors.
- Provides immediate budget status.
- Uses basic Python concepts suitable for learning.
- Can be extended into a complete web application.
- Simple HTML interface is easy to modify.

21. Limitations

- Current Python program accepts total expenses as one value rather than individual transactions.
- Current HTML page is not connected to the Python calculation logic.
- No database is included in the supplied implementation.
- No user authentication is included.
- No bank or payment integration is included.
- Recommendations are rule-based rather than generated by an actual AI model.
- Current version does not automatically maintain monthly history.

• **22. Future Enhancements**

- Connect index.html to a Python web backend using Flask or FastAPI.
- Create a proper budget input form in the browser.
- Add individual expense entries instead of only a total expense value.
- Add categories such as food, travel, education, shopping and bills.
- Store records in SQLite or another database.
- Generate monthly spending charts automatically.
- Add expense history and monthly comparison.
- Add user login and personalized dashboards.
- Add budget-limit alerts.
- Add a recommendation engine using more detailed spending patterns.
- Optionally integrate an AI API for conversational budget assistance, if required.
- Generate downloadable monthly reports.

23. Conclusion

PocketSmart AI: Your Smart Budget & Recommendation and Assistant provides a straightforward approach to monthly budget understanding. The current Python program accepts essential financial inputs, calculates the remaining balance and uses conditional logic to provide a clear recommendation.

The project also includes a basic HTML interface that establishes the PocketSmart AI identity. The architecture, flowchart, data flow diagram, use case diagram and graphical representations show how the current components can be organized into a complete application structure.

Although the supplied implementation is intentionally simple, it provides a clear foundation for future development. Connecting the HTML interface to the Python processing layer, adding expense categories and history, storing data, generating charts and optionally integrating an AI-based assistant can transform the current system into a more comprehensive personal budget management application.

Implementation Note

This documentation describes the supplied PocketSmart AI Python and HTML programs faithfully. Where a diagram or future section describes functionality not present in the supplied code, it is explicitly identified as a proposed extension or illustrative representation.